42TIN1280 Software Analysis (2025-26 - PBTIN)

Wat mag je verwachten van het examen?

Examen Software Analysis – Wat mag je verwachten?

Beste student,
Het examen **Software Analysis** bestaat uit **2 delen**. Samen testen ze of je de leerstof **begrijpt** én kan **toepassen**, zoals geoefend tijdens de lessen.

🧠 Deel 1: Theorie & analyse
🛠️ Deel 2: Praktijk
🚫 Geen giscorrectie (deel 1)
📌 Gebaseerd op leerstof & oefeningen

ℹ️ Algemene info

- Het examen bestaat uit:
  - **Deel 1:** Theorie & analyse
  - **Deel 2:** Praktijk
- **Deel 1** moet eerst worden **afgegeven** voor je aan deel 2 start.
- Voor **deel 1** is er **geen giscorrectie** → lees aandachtig en denk na.
- Alles wat gevraagd wordt, is gebaseerd op de **geziene leerstof en oefeningen**.

🧠 Deel 1 – Theorie & analyse

In dit deel toon je dat je de basisconcepten begrijpt, correct kan toepassen en analytisch nadenkt.

📃 Je mag vragen verwachten over

- **Soorten requirements**
  - business, user stories, functional, non-functional, constraints
- **Kwaliteitsrequirements**
  - herkennen en koppelen aan ISO 25010 (**PERFUMeS**, **PESMURF**)
- **SMART requirements**
  - beoordelen of een requirement SMART is of niet
- **Elicitation**

  - wat het is, welke technieken er zijn, wanneer je welke techniek gebruikt
- **Modellen lezen en interpreteren**
  - contextdiagram
  - domeinmodel
  - SUC-diagram
  - activity diagram
  - state transition diagram en table

👉 Je moet vooral **redeneren, begrijpen en analyseren**, niet definities van buiten leren.

👉 De meerkeuzevragen zijn in de vorm van: **JUIST–FOUT**, **MULTIPLECHOICE**, **SMART** of **NIET-SMART**.

🛠️ Deel 2 – Praktijk

In dit deel pas je Software Analysis toe op een **realistische casus**.

🧩 1. Domeinmodel

- belangrijke concepten uit een casus halen
- een conceptueel domeinmodel tekenen
- correcte relaties (incl. compositie en/of aggregatie), leesrichting en multipliciteiten gebruiken
- een data dictionary invullen (klassen + attributen)

🔄 2. Gedrag van het systeem modelleren

- **Activity diagram**
  - om een proces of workflow te tonen
  - met de juiste volgorde van activiteiten
  - rekening houdend met beslissingen en parallelle stappen
  - over de verschillende swimlanes
- **State transition diagram**
  - om toestanden en overgangen te tonen
  - eventueel aangevuld met een state transition table

👥 3. System Use Case (SUC)

- een SUC-diagram kunnen opstellen
- actoren, use cases en systeemgrens correct bepalen
- rekening houden met generalisaties, includes en extends
- één use case volledig kunnen uitschrijven:

- preconditions
- main success scenario
- alternatieve en exception flows
- postcondition

🗂️ 4. Requirements uitschrijven

- requirements formuleren in natuurlijke taal
- werken met:
  - functionele user stories
  - niet-functionele systeemrequirements
- per requirement: